Prise en main de PHP 8.2

Support de cours et TP (3 h) – version étudiant

Prérequis : HTML, CSS, JavaScript

Environnement : XAMPP, VSCodium

1 PRÉSENTATION DE LA SÉANCE

À la fin des 3 h, vous écrirez seul une page PHP 8.2 qui reçoit un formulaire, valide les données et affiche le résultat en toute sécurité. L'accès aux bases de données sera traité lors de la séance suivante.

Compétences visées

- Expliquer ce qui se passe entre le navigateur et le serveur quand on demande une page `.php`.
- Lancer un projet PHP et mélanger PHP et HTML.
- Manipuler variables, tableaux, boucles et fonctions typées.
- Utiliser quelques apports de PHP 8.x : `declare(strict_types=1)`, `match`, `enum`, promotion de constructeur, classes `readonly` (nouveauté 8.2).
- Traiter un formulaire GET ou POST : lire `$_GET` / `$_POST`, valider, afficher les erreurs, échapper la sortie (faille XSS) et rediriger après un envoi réussi (motif PRG).

Prérequis et matériel

- HTML, CSS et JavaScript (vus précédemment).
- Sur le poste : **XAMPP** (qui fournit PHP), **VSCodium** comme éditeur et un navigateur.
- Aucune base de données n'est nécessaire pendant cette séance.

Déroulé de la séance

Créneau	Durée	Contenu
0 h 00 – 0 h 25	25 min	Partie 1 : fonctionnement de PHP, premier script. Exercice 1
0 h 25 – 1 h 10	45 min	Partie 2 : bases du langage. Exercices 2 et 3
1 h 10 – 1 h 20	10 min	Pause
1 h 20 – 1 h 40	20 min	Partie 3 : PHP 8.2 moderne. Exercice 4
1 h 40 – 2 h 05	25 min	Partie 4 : traiter un formulaire. Exercice 5
2 h 05 – 2 h 55	50 min	Exercice 6 : mini-projet « Inscription à un événement »
2 h 55 – 3 h 00	5 min	Bilan et questions

> Conseil : **retapez** les exemples au lieu de faire des copier-coller. C'est en tapant le code que l'on retient la syntaxe.

2 PARTIE 1 – COMMENT FONCTIONNE PHP

PHP s'exécute **sur le serveur** : le navigateur ne voit jamais le code PHP, seulement le HTML (ou JSON, etc.) qu'il a produit. C'est la différence fondamentale avec le JavaScript que vous connaissez, exécuté dans le navigateur.

2.1 LE CYCLE D'UNE REQUÊTE

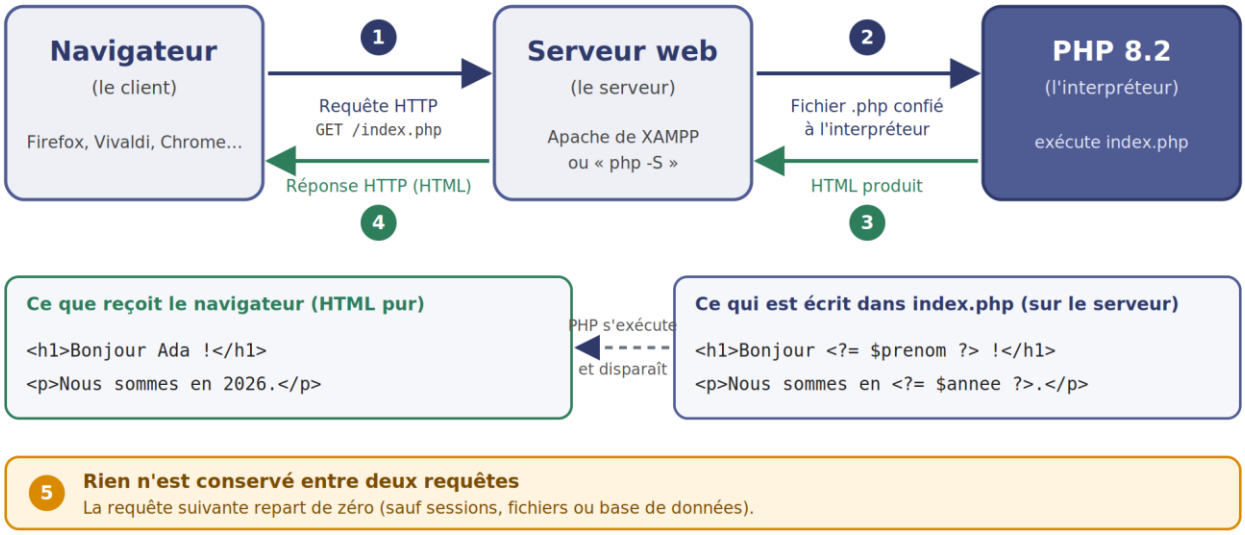


Figure 1 : Le cycle d'une requête

1. Le navigateur envoie une requête HTTP (`GET /index.php`).
2. Le serveur web repère un fichier `.php` et le confie à l'interpréteur PHP.
3. PHP exécute le code ; tout ce qui est hors des balises `<?php ... ?>` est recopié tel quel dans la sortie.
4. Le serveur renvoie le résultat au navigateur, qui l'affiche comme n'importe quelle page HTML.
5. Rien n'est conservé entre deux requêtes, sauf si on le demande explicitement (sessions, fichiers, base de données).

Retenez le dernier point : chaque requête repart de zéro. C'est la source de nombreuses confusions chez les débutants.

2.2 LANCER PHP AVEC XAMPP

XAMPP installe PHP (dans `C:\xampp\php`) et le serveur Apache. Il existe deux façons de s'en servir ; **le TP utilise l'option A**.

Option A (recommandée) : le serveur intégré de PHP

1. Ouvrir le dossier du TP dans VSCodium (menu Fichier, Ouvrir le dossier) : créer d'abord un dossier `tp-php`.
2. Ouvrir un terminal dans VSCodium (menu Terminal, Nouveau terminal). Il est déjà placé dans le dossier du projet.
3. Vérifier la version de PHP :

```
C:\xampp\php\php.exe -v
```

4. Pour pouvoir écrire simplement `php` au lieu du chemin complet, ajouter PHP au chemin de recherche **pour cette fenêtre de terminal** :

```
$env:Path += ";C:\xampp\php"
php -v
```

5. Lancer le serveur depuis le dossier du projet, puis ouvrir `http://localhost:8000/` dans le navigateur :

```
php -S localhost:8000
```

- La première fois, Windows peut demander l'autorisation du pare-feu : l'accepter pour les réseaux privés.
- Le serveur reste actif tant que le terminal est ouvert. On l'arrête avec `Ctrl + C`. Les erreurs PHP s'affichent dans ce terminal.
- Ce serveur est réservé au développement et à l'apprentissage, jamais à la production.

La commande `php -v` doit afficher une version **8.2.x** (ou plus récente). Si XAMPP installe une version plus ancienne, prévenez l'enseignant : les classes `readonly` de la partie 3 exigent PHP 8.2 minimum.

Option B : le serveur Apache de XAMPP

1. Copier le dossier `tp-php` dans `C:\xampp\htdocs\`.
2. Dans le panneau de contrôle XAMPP, cliquer sur **Start** à côté d'Apache.
3. Ouvrir `http://localhost/tp-php/` dans le navigateur.

Si Apache refuse de démarrer, c'est souvent que le port 80 est déjà utilisé par un autre programme : utiliser alors l'option A.

> Dans la suite du support, l'adresse de base est `http://localhost:8000/` (option A). Avec l'option B, remplacez-la par `http://localhost/tp-php/`.

2.3 PREMIER SCRIPT

```php
<?php
declare(strict_types=1);


$prenom = 'Ada';
$annee  = (int) date('Y');
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Bonjour PHP</title></head>
<body>
  <h1>Bonjour <?= $prénom ?> !</h1>
  <p>Nous sommes en <?= $année ?>.</p>
</body>
</html>
```

- `<?php ... ?>` ouvre et ferme un bloc de code ; `<?= expr ?>` est le raccourci pour afficher une valeur.
- Un fichier qui ne contient que du PHP n'a pas besoin de balise fermante `?>`.
- Pour tester du code sans page web : `php fichier.php` dans le terminal.
- Pendant le TP, on peut afficher toutes les erreurs en début de script avec `error_reporting(E_ALL);` et `ini_set('display_errors', '1');`.

3 PARTIE 2 – BASES DU LANGAGE

Quand on connaît JavaScript, PHP s'apprend surtout par comparaison : la logique est la même, la syntaxe change sur quelques points.

Concept	JavaScript	PHP
Variable	`let x = 1;`	`$x = 1;`
Concaténation	`a + b`	`$a . $b`
Texte avec variable	`` `Salut ${nom}` ``	`"Salut $nom"` ou `"Salut {$user['nom']}"`
Taille d'un tableau	`arr.length`	`count($arr)`
Tableau associatif / objet littéral	`{ nom: 'Ada' }`	`['nom' => 'Ada']`
Accès à une propriété d'objet	`obj.nom`	`$obj->nom`
Fonction fléchée	`x => x * 2`	`fn($x) => $x * 2`
Valeur par défaut si null	`a ?? b`	`$a ?? $b`
Affichage / débogage	`console.log(x)`	`echo $x;` / `var_dump($x);`

3.1 VARIABLES ET TYPES

```php
<?php
 $nom    = 'Ada';          // string
 $age    = 36;             // int
 $taille = 1.68;           // float
 $actif  = true;           // bool
 $rien   = null;           // null

 echo "Bonjour $nom, vous avez $age ans.\n";   // guillemets doubles : interpolation
 echo 'Bonjour $nom';                          // guillemets simples : texte brut
 var_dump($age, $taille, $actif);              // affiche le type ET la valeur
```

Points d'attention :

- Préférez toujours `===` à `==`. Avec `==`, PHP convertit les types avant de comparer, ce qui donne des surprises (par exemple `'1' == '01'` vaut `true`).
- `??` renvoie la valeur de droite si celle de gauche est `null` **ou n'existe pas** ; c'est l'outil de base pour lire `$_GET` et `$_POST` sans erreur.
- `isset($x)` teste qu'une variable existe et n'est pas `null`. `empty($x)` considère aussi `0`, `''`, `'0'` et `[]` comme vides (attention : `empty('0')` vaut `true`).

3.2 STRUCTURES DE CONTRÔLE

```php
<?php
$note = 8 ;
if ($note >= 10) {
    echo 'Admis';
} elseif ($note >= 8) {
    echo 'Rattrapage' . "<br>";
} else {
    echo 'Ajourné' . "<br>";
}
```

```
for ($i = 1; $i <= 3; $i++) {
    echo "Ligne $i <br>";
}
```

3.3 TABLEAUX : LA STRUCTURE CENTRALE DE PHP

Le type `array` joue à la fois le rôle de tableau et de dictionnaire.

```
<?php
$langages = ['PHP', 'JavaScript', 'Python'];          // tableau indexé
$etudiant = ['nom' => 'Ada', 'note' => 17.5];          // tableau associatif

$langages[] = 'Rust';                                  // ajout en fin de tableau
echo $etudiant['nom'];                                 // accès par clé

foreach ($langages as $index => $langage) {
    echo "$index : $langage <br>";
}

foreach ($etudiant as $cle => $valeur) {
    echo "$cle = $valeur <br>";
}

$notes = [12, 15, 9, 18];
echo count($notes);                                    // 4
echo array_sum($notes) / count($notes);                // 13.5
echo implode(', ', $notes);                            // "12, 15, 9, 18"
var_dump(in_array(15, $notes, true));                  // true
$doubles = array_map(fn(int $n): int => $n * 2, $notes);
```

Fonctions de tableau à connaître pour la suite du TP : `count`, `in_array`, `array_key_exists`, `array_map`, `array_filter`, `array_sum`, `implode`, `explode`, `sort`.

3.4 FONCTIONS

```
<?php
declare(strict_types=1);

function moyenne(array $notes): float
{
    if ($notes === []) {
        return 0.0;
    }
    return array_sum($notes) / count($notes);
}

function saluer(string $nom, string $politesse = 'Bonjour'): string
{
    return "$politesse $nom";
}

echo saluer('Ada');                  // Bonjour Ada
echo moyenne([12, 15, 9]);           // 12
```

Avec `declare(strict_types=1);` en première instruction du fichier, PHP **refuse** les conversions implicites sur les appels de fonction : `saluer(42)` provoque une `TypeError` au lieu de convertir silencieusement 42 en texte. C'est la bonne pratique à adopter dès le premier jour.

4 PARTIE 3 – PHP 8.2 MODERNE

PHP a beaucoup changé depuis la version 7 : on peut aujourd'hui écrire un code typé et concis, bien loin de l'image du « vieux PHP ». Cette partie présente uniquement ce qui sert dans le TP, pas un catalogue complet.

Fonctionnalité	Depuis	Intérêt
Arguments nommés	8.0	Appels lisibles : `new Etudiant(nom: 'Ada', note: 15.5)`
Promotion de constructeur	8.0	Déclare et affecte les propriétés directement dans `__construct`
Expression `match`	8.0	Un `switch` strict qui renvoie une valeur
Opérateur null-safe `?->`	8.0	`$user?->adresse?->ville` renvoie `null` au lieu de planter
Types union (`int|string`)	8.0	Types plus précis pour paramètres et retours
`str_contains`, `str_starts_with`, `str_ends_with`	8.0	Remplacent les `strpos(...) !== false`
Énumérations (`enum`)	8.1	Ensemble fini de valeurs typées (statuts, rôles, mentions)
Propriétés `readonly`	8.1	Valeur affectée une seule fois, puis immuable
Classes `readonly`	**8.2**	Toutes les propriétés de la classe sont `readonly`
Types `null`, `false`, `true` autonomes	8.2	Types de retour plus fins
Propriétés dynamiques dépréciées	8.2	Créer `$obj->truc` non déclaré émet un avertissement de dépréciation
Interpolation `"${var}"` dépréciée	8.2	Écrire `"$var"` ou `"{$var}"`

4.1 EXEMPLE FIL ROUGE : ENUM + CLASSE READONLY + MATCH

```php
<?php
declare(strict_types=1);

enum Mention: string
{
    case Ajourne  = 'Ajourné';
    case Passable = 'Passable';
    case Bien     = 'Bien';
    case TresBien = 'Très bien';

    public static function depuisNote(float $note): self
    {
```

```
        return match (true) {
            $note >= 16 => self::TresBien,
            $note >= 14 => self::Bien,
            $note >= 10 => self::Passable,
            default     => self::Ajourne,
        };
    }
}

final readonly class Etudiant
{
    public function __construct(
        public string $nom,
        public float $note,
    ) {}

    public function mention(): Mention
    {
        return Mention::depuisNote($this->note);
    }
}

$ada = new Etudiant(nom: 'Ada', note: 15.5);
echo $ada->nom . ' : ' . $ada->mention()->value . "\n";   // Ada : Bien

$ada->note = 12;   // Error : Cannot modify readonly property Etudiant::$note
```

À retenir :

- `match` compare en **strict** (`===`), renvoie une valeur et lève une `UnhandledMatchError` si aucun cas ne correspond (d'où le `default`).
- Un `enum` « backed » (`: string`) dispose de `Mention::from('Bien')` (erreur si la valeur n'existe pas) et de `Mention::tryFrom('Bien')` (renvoie `null` si elle n'existe pas). `tryFrom` est très utile pour valider une liste déroulante dans un formulaire.
- Une classe `readonly` ne peut ni être modifiée après construction, ni recevoir de propriété dynamique : les données gardent un état prévisible, ce qui est précieux pour des objets qui transportent des données saisies par l'utilisateur.
- Ici, la classe sert simplement de « conteneur de données » typé. La programmation orientée objet n'est pas l'objet de cette séance.

5 PARTIE 4 – TRAITER UN FORMULAIRE

Tout ce qui arrive d'un formulaire est une donnée **non fiable** : PHP la reçoit sous forme de texte, elle peut être absente, mal formée ou malveillante. Traiter un formulaire, c'est donc toujours enchaîner quatre étapes : recevoir, valider, traiter, afficher en échappant.

5.1 GET OU POST ?

	GET	POST
Où voyagent les données	Dans l'URL (`?q=php&page=2`)	Dans le corps de la requête
Tableau PHP	`$_GET`	`$_POST`
Usage typique	Recherche, filtre, pagination (action sans effet de bord, lien partageable)	Inscription, connexion, création ou modification de données
Rechargement de la page	Sans risque	Le navigateur demande de renvoyer le formulaire

Pour savoir quelle méthode a été utilisée : `$_SERVER['REQUEST_METHOD']` vaut `'GET'` ou `'POST'`.

5.2 LES QUATRE RÉFLEXES

1. **Ne jamais supposer qu'une clé existe** : `$_POST['nom'] ?? ''`. Un client peut très bien envoyer la requête sans ce champ.
2. **Ne jamais supposer que c'est du texte** : un attaquant peut envoyer `nom[]=x`, ce qui donne un tableau. On vérifie avec `is_string()`.
3. **Valider côté serveur** : les attributs HTML `required`, `type="email"` ou `min` améliorent l'ergonomie, mais se contournent en deux secondes (outils de développement, `curl`). Ils ne protègent de rien.
4. **Échapper à l'affichage** : toute donnée saisie par un utilisateur et réaffichée dans une page passe par `htmlspecialchars()`. Sans cela, un visiteur qui saisit `<script>alert(1)</script>` fait exécuter son code dans le navigateur des autres : c'est la faille **XSS**.

5.3 EXEMPLE COMPLET : UN FORMULAIRE QUI SE TRAITE LUI-MÊME

```php
<?php
declare(strict_types=1);

function h(string $texte): string
{
    return htmlspecialchars($texte, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}

function champ(string $cle): string
{
    $valeur = $_POST[$cle] ?? '';
    return is_string($valeur) ? trim($valeur) : '';
}
```

```
$erreurs = [];
$message = null;
$nom   = '';
$email = '';
$age   = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nom   = champ('nom');
    $email = champ('email');
    $age   = champ('age');

    if ($nom === '') {
        $erreurs['nom'] = 'Le nom est obligatoire.';
    } elseif (mb_strlen($nom) > 50) {
        $erreurs['nom'] = 'Le nom ne doit pas dépasser 50 caractères.';
    }

    if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
        $erreurs['email'] = 'Adresse e-mail invalide.';
    }

    $ageValide = filter_var($age, FILTER_VALIDATE_INT, [
        'options' => ['min_range' => 16, 'max_range' => 99],
    ]);
    if ($ageValide === false) {
        $erreurs['age'] = 'L\'âge doit être un entier entre 16 et 99.';
    }

    if ($erreurs === []) {
        // Données valides : on peut les traiter (la base de données viendra à la
séance suivante).
        $message = 'Merci ' . $nom . ' !';
    }
}
?>
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Inscription</title></head>
<body>
  <?php if ($message !== null): ?>
    <p><?= h($message) ?></p>
  <?php endif; ?>
  <form method="post" action="inscription.php" novalidate>
    <p>
      <label>Nom <input type="text" name="nom" value="<?= h($nom) ?>"></label>
      <?php if (isset($erreurs['nom'])): ?>
        <span style="color:#b00"><?= h($erreurs['nom']) ?></span>
      <?php endif; ?>
    </p>
    <!-- idem pour email et age -->
    <button type="submit">S'inscrire</button>
  </form>
</body>
</html>
```

À observer dans cet exemple :

- Le formulaire se renvoie vers la **même page** : le test `REQUEST_METHOD === 'POST'` distingue l'affichage initial (GET) de la soumission (POST).
- En cas d'erreur, on **réaffiche les valeurs saisies** (`value="<?= h($nom) ?>"`), pour que l'utilisateur ne retape pas tout.
- `novalidate` désactive la validation du navigateur pendant le TP, afin de pouvoir tester la validation PHP.
- Syntaxe alternative `if (...): ... endif;` : plus lisible que les accolades quand on alterne PHP et HTML.

5.4 LE MOTIF PRG (POST / REDIRECT / GET)

Si, après un POST réussi, on affiche directement une page de confirmation, un rechargement (F5) renvoie le formulaire et refait l'action (double inscription). La parade : après un traitement réussi, rediriger vers une page en GET.

```
<?php
session_start();
// ... validation réussie ...
$_SESSION['message'] = 'Inscription enregistrée.';
header('Location: inscription.php');
exit;   // indispensable : stoppe le script juste après la redirection
```

Sur la page de destination, on lit le message puis on le supprime pour qu'il ne s'affiche qu'une fois :

```
<?php
session_start();
$message = $_SESSION['message'] ?? null;
unset($_SESSION['message']);
```

`header()` doit être appelé **avant toute sortie** (aucun `echo`, aucun HTML, même une ligne vide avant `<?php`). `$_SESSION` est un tableau qui persiste d'une requête à l'autre pour un même visiteur : on l'utilise ici comme une « boîte noire », sans en détailler le fonctionnement.

Hors du périmètre de cette séance, mais à connaître : la protection CSRF, le téléversement de fichiers et l'authentification.

6 TP – PRISE EN MAIN DE PHP 8.2

Ce TP comporte six exercices progressifs ; le dernier réunit tout ce qui précède dans un petit projet de formulaire.

6.1 PRÉPARATION (5 MIN)

1. Créer un dossier `tp-php` et l'ouvrir dans VSCodium (Fichier, Ouvrir le dossier).
2. Ouvrir un terminal (Terminal, Nouveau terminal) et vérifier la version : `php -v` doit afficher `PHP 8.2.x` (voir la partie 1 pour ajouter PHP au chemin de recherche).
3. Lancer le serveur intégré depuis ce dossier : `php -S localhost:8000`. Le laisser tourner pendant tout le TP.
4. Ouvrir `http://localhost:8000/` dans le navigateur. Une erreur 404 est normale tant qu'aucun fichier n'existe.

Avec l'option B (Apache de XAMPP), le dossier se trouve dans `C:\xampp\htdocs` et l'adresse de base est `http://localhost/tp-php/`.

Règles de la séance : chaque fichier commence par `<?php` suivi de `declare(strict_types=1);` ; on **retape** le code des exemples ; on teste dans le navigateur après chaque modification (touche F5).

6.2 EXERCICE 1 – PREMIER SCRIPT (10 MIN)

Objectif : voir de ses yeux que PHP s'exécute côté serveur.

- Créer `index.php`, qui affiche une page HTML complète avec un titre `Bonjour <prénom>` (le prénom vient d'une variable PHP) et la date du jour au format `14/10/2026` (fonction `date('d/m/Y')`).
- Ajouter une liste `<ul>` de vos trois langages préférés, générée avec un tableau PHP et une boucle `foreach`.
- Dans le navigateur, faire un clic droit puis « Afficher le code source de la page ».

À vérifier

- ☐ La page s'affiche sans erreur et la date est correcte.
- ☐ Le code source ne contient **aucune** balise `<?php`. Pourquoi ?
- ☐ Dans un second terminal, `php index.php` affiche le HTML en texte : en quoi diffère-t-il de ce que reçoit le navigateur ?

6.3 EXERCICE 2 – TABLEAUX ET BOUCLES (20 MIN)

Objectif : produire un tableau HTML à partir d'un tableau PHP.

- Créer `notes.php` avec ce tableau associatif :

```
$notes = [
    'Ada'      => 17.5,
    'Alan'     => 14.0,
    'Grace'    => 9.5,
    'Linus'    => 12.0,
    'Margaret' => 6.5,
];
```

- Afficher un `<table>` HTML avec une ligne par étudiant (nom, note) construite avec `foreach ($notes as $nom => $note)`.
- Colorer en rouge la ligne des notes strictement inférieures à 10 (une classe CSS `echec` ajoutée conditionnellement).
- Sous le tableau, afficher la moyenne (arrondie à 2 décimales avec `round()`), la note minimale et la note maximale (`min()`, `max()`, `array_sum()`, `count()`).

À vérifier

- ☐ Résultat attendu : moyenne `11.9`, minimum `6.5`, maximum `17.5`.
- ☐ Ajouter un sixième étudiant dans le tableau le fait apparaître sans toucher au reste du code.

Indice : `<tr class="<?= $note < 10 ? 'echec' : '' ?>">`.

6.4 EXERCICE 3 – FONCTIONS TYPÉES (20 MIN)

Objectif : organiser le code en fonctions réutilisables dans un autre fichier.

- Créer `fonctions.php` (avec `declare(strict_types=1);`) contenant :
  - `moyenne(array $notes): float` : renvoie `0.0` si le tableau est vide ;
  - `estAdmis(float $note, float $seuil = 10.0): bool` ;
  - `meilleurEtudiant(array $notes): ?string` : renvoie le nom de l'étudiant qui a la meilleure note, ou `null` si le tableau est vide (indice : `array_search`, `max`).
- Dans `notes.php`, charger ce fichier avec `require_once __DIR__ . '/fonctions.php';` et remplacer vos calculs de l'exercice 2 par des appels à ces fonctions.
- Afficher « Meilleur étudiant : Ada » sous le tableau, avec un texte de repli si `meilleurEtudiant()` renvoie `null` (opérateur `??`).

À vérifier

- ☐ `notes.php` donne les mêmes résultats qu'à l'exercice 2.
- ☐ Appeler `estAdmis('12')` (avec des guillemets) provoque une `TypeError` : expliquer pourquoi, puis retirer cet appel de test.
- ☐ Tester `moyenne([])` : pas de division par zéro.

6.5 EXERCICE 4 – PHP 8.2 MODERNE : ENUM, MATCH ET CLASSE READONLY (20 MIN)

Objectif : modéliser un étudiant avec des types précis.

- Dans un nouveau fichier `modele.php`, reprendre l'exemple fil rouge du cours : `enum Mention` (valeurs `Ajourné`, `Passable`, `Bien`, `Très bien`) et `final readonly class Etudiant` avec promotion de constructeur (`nom`, `note`) et une méthode `mention(): Mention`.
- Dans `notes.php`, transformer le tableau `$notes` en un tableau d'objets `Etudiant` (en utilisant des **arguments nommés** à la construction) et ajouter une colonne « Mention » au tableau HTML.
- Ajouter une méthode `estAdmis(): bool` à la classe, qui utilise `match` ou une comparaison sur la mention.
- Tenter de modifier `$etudiant->note = 20;` dans un bloc `try { ... } catch (Error $e) { echo htmlspecialchars($e->getMessage()); }` pour voir l'erreur levée par `readonly`.

À vérifier

- ☐ La colonne « Mention » s'affiche correctement pour chaque étudiant.
- ☐ Le message d'erreur sur la propriété `readonly` est affiché proprement.
- ☐ Question : que se passe-t-il si on écrit `$etudiant->age = 20;` sur une classe `readonly` ? et sur une classe normale en PHP 8.2 ?

6.6 EXERCICE 5 – PREMIERS FORMULAIRES (20 MIN)

Objectif : lire `$_GET` / `$_POST` et comprendre pourquoi il faut échapper.

- **Partie A (GET)** : créer `salut.php` avec un formulaire `method="get"` contenant un champ `prenom`. Quand le champ est renseigné, afficher « Bonjour <prénom> ! », sinon n'afficher que le formulaire. Observer l'URL après soumission.
- **Partie B (attaque XSS)** : saisir `<b>Ada</b>` puis `<script>alert('XSS')</script>` **sans** échapper la sortie. Que se passe-t-il ? Corriger avec `htmlspecialchars($prenom, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')` et retester.
- **Partie C (POST)** : passer le formulaire en `method="post"` et lire `$_POST`. Observer ce qui change dans l'URL et dans l'onglet Réseau des outils de développement du navigateur (touche F12).
- Dans les trois cas, lire les données avec `??` pour qu'aucune alerte « undefined array key » n'apparaisse au premier affichage.

À vérifier

- ☐ Avant correction, la balise `<script>` s'exécute ; après correction, elle est affichée comme du texte.
- ☐ Aucun avertissement PHP au premier chargement de la page.
- ☐ Vous savez expliquer quand préférer GET, quand préférer POST.

6.7 EXERCICE 6 – MINI-PROJET « INSCRIPTION À UN ÉVÉNEMENT » (50 MIN)

Objectif : réaliser seul un formulaire complet, validé côté serveur, sans base de données.

Créer `inscription.php`, avec un formulaire d'inscription à un hackathon, composé des champs suivants :

Champ	Type HTML	Règles de validation
Nom	texte	Obligatoire, 2 à 50 caractères
E-mail	`email`	Obligatoire, adresse valide (`FILTER_VALIDATE_EMAIL`)
Âge	nombre	Entier entre 16 et 99
Filière	liste déroulante	Valeur parmi une liste autorisée (INFO, MATH, PHYS, AUTRE) ; utiliser un `enum` et `tryFrom()`
Langages maîtrisés	cases à cocher (`name="langages[]"`)	Chaque valeur appartient à la liste autorisée ; au moins un choix
Message	zone de texte	Facultatif, 300 caractères maximum
Règlement	case à cocher	Obligatoire

Exigences

- Un seul fichier gère l'affichage initial (GET) et la soumission (POST).
- Chaque champ en erreur affiche son propre message, à côté du champ.
- Les valeurs saisies sont **réaffichées** après une erreur (y compris les cases cochées et la liste déroulante).
- Toute donnée réaffichée est échappée avec `htmlspecialchars` (écrire une fonction `h()`).
- Si tout est valide : stocker un message de confirmation en session, **rediriger** vers `inscription.php` (motif PRG), puis afficher le message une seule fois.
- Le code est découpé en fonctions courtes et typées (`declare(strict_types=1);`), dans un fichier `fonctions.php` réutilisé.

À vérifier

- ☐ Un formulaire vide affiche des erreurs pour tous les champs obligatoires.
- ☐ Un nom `<script>alert(1)</script>` ne s'exécute pas, ni dans le champ, ni dans le message.
- ☐ Un âge de `abc`, `15` ou `12.5` est refusé ; `16` est accepté.
- ☐ Après une inscription réussie, le rechargement de la page (F5) ne renvoie pas le formulaire.
- ☐ En modifiant la requête avec les outils du navigateur (par exemple filière = `HACK`), le serveur refuse la valeur.

Pour aller plus loin (bonus)

- Refuser deux inscriptions avec la même adresse e-mail.
- Ajouter un style CSS (vous connaissez déjà le sujet).

7 ANNEXES

7.1 AIDE-MÉMOIRE

Besoin	Code
Afficher une valeur dans le HTML	`<?= $x ?>`
Déboguer une variable	`var_dump($x);`
Lire un champ sans avertissement	`$_POST['nom'] ?? ''`
Savoir si le formulaire a été envoyé	`$_SERVER['REQUEST_METHOD'] === 'POST'`
Échapper avant d'afficher	`htmlspecialchars($t, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')`
Valider un entier dans une plage	`filter_var($v, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1, 'max_range' => 9]])`
Valider un e-mail	`filter_var($v, FILTER_VALIDATE_EMAIL)`
Tester la présence d'une valeur dans une liste	`in_array($v, $liste, true)`
Inclure un fichier une seule fois	`require_once __DIR__ . '/fonctions.php';`
Rediriger	`header('Location: page.php'); exit;`
Typage strict	`declare(strict_types=1);` en première instruction du fichier

7.2 ERREURS FRÉQUENTES ET DIAGNOSTIC

Symptôme	Cause probable	Solution
`php` n'est pas reconnu comme commande	PHP de XAMPP absent du chemin de recherche	Taper `$env:Path += ";C:\xampp\php"` ou utiliser `C:\xampp\php\php.exe`
Le navigateur affiche le code PHP ou propose de télécharger le fichier	Page ouverte avec `file://` ou serveur non lancé	Utiliser `http://localhost:8000/fichier.php`
Apache ne démarre pas dans XAMPP	Port 80 déjà utilisé	Utiliser le serveur intégré (option A)
Page blanche	Erreur fatale non affichée	Lire le terminal du serveur ; ajouter `error_reporting(E_ALL); ini_set('display_errors', '1');`
`Cannot modify header information – headers already sent`	Du texte ou du HTML a été envoyé avant `header()` (y compris un espace avant `<?php`)	Faire la logique PHP **avant** tout HTML ; ne pas fermer un fichier de pur PHP par `?>`
`Undefined array key "nom"`	Lecture directe de `$_POST['nom']` alors que la clé n'existe pas	Utiliser `?? ''`
`TypeError … must be of type float, string given`	Avec `strict_types`, les valeurs de formulaire sont toujours des chaînes	Valider avec `filter_var`, puis convertir explicitement
Accents illisibles (Ã©)	Fichier non enregistré en UTF-8 ou `<meta charset>` absent	Enregistrer en UTF-8 et déclarer `<meta charset="utf-8">`
Une modification n'apparaît pas	Mauvais dossier pour `php -S` ou cache du navigateur	Relancer le serveur depuis le bon dossier ; `Ctrl + F5`
`Failed to listen … Address already in use`	Un serveur occupe déjà le port	Choisir un autre port : `php -S localhost:8001`
`Class "Etudiant" not found`	Fichier non inclus	`require_once __DIR__ . '/modele.php';`
`Cannot redeclare function …`	Fichier de fonctions inclus deux fois avec `require`	Utiliser `require_once`

7.3 POUR ALLER PLUS LOIN

- **Séance suivante :**
  - Accès à une base de données avec PDO, requêtes préparées (protection contre l'injection SQL).
- **Références :**
  - Le manuel officiel php.net/manual/fr ;
  - les notes de version PHP 8.2 sur php.net/releases/8.2 ;
  - Le guide « PHP: The Right Way » (phptherightway.com) ;
  - et 3v4l.org pour comparer le comportement d'un script entre versions de PHP.